CS 336: Assignment 1

Author

April 15, 2025

Contents

2. BPE Tokenizer

2.1

Problem (`unicode1`): Understanding Unicode (1 point)

(a) ‘x00’ null character (b) Same just escaped. repr(chr(0)) result is “‘\x00’” (c) It’s skipped in string. Python it is treated as a valid, invisible character that does not truncate the text, allowing the rest of the string to print normally.

2.2 (a) Save space coz utf-8 char are most frequent (b) UTF-8 is variable length (c) b’xffxff’

2.4 train_bpe_tinystories (a) Total execution time: 630.48 seconds Peak memory usage: 109.06 MB Longest token is b’ accomplishment’. Makes sense —Indexed bp impl Total execution time: 337.36 seconds Peak memory usage: 104.03 MB Longest token is b’ accomplishment’. Makes sense (b) Pre-tokenization took 184.20 seconds. Merging took 446.24 seconds. —Indexed bp impl Pre-tokenization took 150.08 seconds. Merging took 187.25 seconds.

2.7 (a) TinyStories: 4.15 bytes/token owt: 4.51 bytes/token (b) 3.40 bytes/token. Compression ration dropped (c) Throughput: 811635.81 bytes/second. `825*1024*1024*1024/811635.81/3600 = 303.17 hour = 12.6 days` (d) uint16 has range of 0-65535, which works for vocab size 10000 or 32000

3

3.6 The Full Transformer LM

Problem (transformer_accounting): Transformer LM resource accounting (5 points)

a) Expression for total trainable parameters Trainable parameters per transformer block:

$$\begin{aligned} P_{\text{transformer\_block}} &= 2 \text{ RMSNorm} + \text{QKVO proj} + \text{FFN} \\ &= 2d_m + 4d_m^2 + 3d_m d_{\text{ff}} \end{aligned} \tag{1}$$

Total trainable parameters:

$$\begin{aligned} P_{\text{total}} &= P_{\text{transformer\_block}} \times \text{num\_layers} + \text{down/up proj (aka token/output emb)} \\ &+ \text{Final RMSNorm} \\ &= \left(2d_m + 4d_m^2 + 3d_m d_{\text{ff}}\right) \times \text{num\_layers} + 2d_m \text{ vocab\_size} + d_m \\ &= 2,127,057,600 \end{aligned} \tag{2}$$

Memory needed to load the model `2,127,057,600 * 4 / 1024/1024/1024 = 7.92 GB`

b) MatMuls: QKVO, Attention, FFN. token/output embedding are not real MatMul because it's one-hot

QKVO projection FLOPs:

$$\text{FLOP}_{\text{qkvo}} = 4 \times 2ld^2 = 8ld^2 \tag{3}$$

Attention FLOPs:

$$\text{FLOP}_{\text{attn}} = 2 \times 2l^2 d = 4l^2 d \tag{4}$$

FFN FLOPs:

$$\text{FLOP}_{\text{ffn}} = 3 \times 2ldd_{\text{ff}} = 6ldd_{\text{ff}} \tag{5}$$

Each transformer block FLOPs:

$$\begin{aligned} \text{FLOP}_{\text{transformer\_block}} &= \text{FLOP}_{\text{qkvo}} + \text{FLOP}_{\text{attn}} + \text{FLOP}_{\text{ffn}} \\ &= 8ld^2 + 4l^2 d + 6ldd_{\text{ff}} \end{aligned} \tag{6}$$

Total FLOPS from MatMuls

$$\begin{aligned} \text{FLOP}_{\text{total}} &= \text{FLOP}_{\text{transformer\_block}} \times \text{num\_layers} \\ &= \left(8ld^2 + 4l^2 d + 6ldd_{\text{ff}}\right) \times \text{num\_layers} \end{aligned} \tag{7}$$

Plug in l=1024 d=1600 d_ff=6400 num_layers=48

$$\begin{aligned} &\text{FLOP}_{\text{total}} \\ &= 48 \times (8 \times 1024 \times 1600 \times 1600 + 4 \times 1024 \times 1024 \times 1600 + 6 \times 1024 \times 1600 \times 6400) \\ &= 4,348,654,387,200 \approx 4.38 \text{ TFlops} \end{aligned} \tag{8}$$

`48 * (8*1024*1600*1600 + 4*1024*1024*1600 + 6*1024*1600*6400) to avoid retyping`

c)

Inside transformer block, FFN (d) Parameters GPT-2 small GPT-2 small % GPT-2 medium GPT-2 medium % GPT-2 large GPT-2 large % FLOP of each parts QKVO projection 4,831,838,208 1.38% 8,589,934,592 0.83% 13,421,772,800 0.59% MHA 3,221,225,472 0.92% 4,294,967,296 0.42% 5,368,709,120 0.24% FFN 14,495,514,624 4.15% 25,769,803,776 2.49% 40,265,318,400 1.78% Total Transformer Block 270,582,939,648 77.39% 927,712,935,936 89.80% 2,126,008,811,520 94.16% Output Emb 79,047,426,048 22.61% 105,396,568,064 10.20% 131,745,710,080 5.84% Total LM 349,630,365,696 1,033,109,504,000 2,257,754,521,600 Observation: QKVO, FFN increase as model increase, MHA and output emb decreased (e) FLOP of 1 forward pass x 33 Now MHA accounts for most of the FLOP, where as other components dropped 4.2 Experiments Observations: lr=1: loss gradually decreased, from 27 to 23 lr=1e1: loss more quickly decreased from 29 to 4 lr=1e2: loss drastically decreased from 31 to e-23 lr=1e3: loss diverged Answer: Lower learning rates (lr=1) caused slow, steady loss decay, while moderate rates (lr=1e1, 1e2) decayed faster — with lr=1e2 reaching near zero within 10 steps. lr=1e3 caused the loss to diverge, increasing rather than decreasing. 4.3 (a) B=batch_size, L=context_length, d=d_model, V=vocab_size, n=num_layers, H=num_heads d_ff = 8/3 d_model P = $n(16d^2 + 2d) + d + 2Vd \approx 16nd^2 + 2Vd$ Gradients = P optimizer = 2P (AdamW) Activations = n(16BLd + 2BHL^2) + BLd (final RMS) + BLV (logits) Total = 16nd^2 + 2nd + d + 2Vd + n(16BLd + 2BHL^2) + BLd + BLV (b) 14.26 B + 31.7 max_b = 3 (c) 13P = 208nd^2 + 26nd + 13d + 26Vd Each step of AdamW takes m 3 / v 4 / update parameter 6 steps each. Totalling 13P flops (d) `From 3.6, each forward step is 4.51*1024 = 4618 TFLOPs. Backward 9.02*1024=9237 TFLOPs` B=1024 n=48, d=1600, V=50257 Optimizer flops are `13*(48*(16*1600*1600+2*1600) +1600+2*1600*50257) = 27,651,748,800 ~ 0.28 TFLOPs` Total FLOPS per step = 13855 TFLOPs `400000*13855/19.5/0.5/24/3600/365=18 years`